

Architectural Tradeoffs in Low-Latency Algorithmic Trading with Predictive Signals

Abhinav Randive

February 19, 2026

1 Introduction

The introduction will broadly introduce your topic, motivate it, and describe what will be done.

2 Related Works

3 Methodology

This project begins by constructing the foundational infrastructure required for an event-driven trading system simulation. At this stage, the focus is on market data ingestion and deterministic event replay, which form the basis for analyzing architectural tradeoffs in low-latency trading systems.

3.1 Market Data Acquisition

Historical market data is downloaded using the Yahoo Finance API through the `yfinance` Python library. The current implementation retrieves recent S&P 500 index data at one-minute intervals and stores timestamped price and volume information in CSV format. The preprocessing step flattens multi-index columns, standardizes column names, and converts timestamps to integer Unix time for consistent processing.

This approach ensures reproducibility and establishes a structured input format for the simulation pipeline. Using historical replay rather than live feeds aligns with common practices in quantitative finance research, where controlled backtesting environments are used to evaluate trading logic [1, 5].

3.2 Event Abstraction

To support an event-driven architecture, market data is abstracted into discrete events. An `Event` data structure is defined with three fields: timestamp, event type, and payload.

Event types are enumerated to allow future extension for order submission, cancellation, and execution events.

Each market update is represented as a structured event object, enabling uniform handling of data within the system. This abstraction follows principles of real-time stream processing systems, where events are processed sequentially and deterministically [4]. Event abstraction is critical for modeling realistic trading environments, where exchanges operate as event-driven systems governed by limit order book mechanics [3].

3.3 Deterministic Market Replay

A replay engine iterates sequentially over historical market data and converts each row into a `MARKET_UPDATE` event. The replay mechanism maintains an internal index and exposes methods to check for remaining events and retrieve the next event in chronological order.

Deterministic replay prevents look-ahead bias and ensures that events are processed strictly in historical sequence. This is essential when studying the interaction between prediction and execution, since improper ordering can artificially inflate model performance [2]. Although a full event dispatcher and execution engine have not yet been implemented, the replay system establishes the primary input stream for the future low-latency architecture.

4 Results

An exemplary figure is shown in Figure 1. Any time a figure is used, it must have a caption and be directly referred to and discussed in the text.

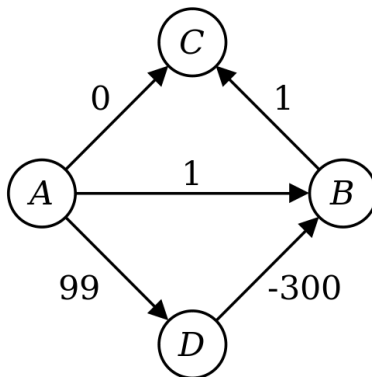


Figure 1: Dijkstra’s algorithm does not work on graphs with negative edge weights, as evidenced here.

5 Conclusion

References

- [1] ALDRIDGE, I. *High-Frequency Trading: A Practical Guide to Algorithmic Strategies and Trading Systems*, 2nd ed. John Wiley & Sons, 2013.
- [2] BOUCHAUD, J.-P., FARMER, J. D., AND LILLO, F. How markets slowly digest changes in supply and demand. *Handbook of Financial Markets: Dynamics and Evolution* (2009), 57–160.
- [3] HARRIS, L. *Trading and Exchanges: Market Microstructure for Practitioners*. Oxford University Press, 2003.
- [4] STONEBRAKER, M., ÇETINTEMEL, U., AND ZDONIK, S. The 8 requirements of real-time stream processing. *Association for Computing Machinery Special Interest Group on Management of Data Record* 34, 4 (2005), 42–47.
- [5] ÁLVARO CARTEA, JAIMUNGAL, S., AND PENALVA, J. *Algorithmic and High-Frequency Trading*. Cambridge University Press, 2015.